OR Deliverable 1: CNN architecture design

March 2nd, 2026, By Daniel Hess, Emre Karaoglu, Elias Miguel Leal Spohr, Lukas Gabriel Sekinger, Pandelis Laurens Symeonidis

Introduction

Multi-label image classification presents new challenges compared to single-label tasks, specifically when there is class imbalance, scale variation, and overlapping objects. The Pascal VOC 2012 dataset shows these difficulties: images can contain multiple objects, object scales vary a lot between classes, and certain image classes dominate the dataset distribution.

The goal of this work is to systematically design and evaluate CNN architectures for multi-label classification under controlled settings. Rather than focusing on maximizing accuracy, we try to find the trade-offs between model capacity, computational efficiency, convergence behavior, and generalization performance. Each task builds upon the previous one, beginning with dataset analysis, followed by a baseline implementation, architecture comparison, transfer learning optimization, and finally supervision and loss design improvements.

Task 1 - Dataset Analysis

The Pascal VOC 2012 train/val set contains 17,125 images with 40,138 annotated object instances across 20 classes. Table 1 summarises the per-class statistics for all three sub-tasks: (a) object instance counts, (b) image-level counts, and (c) bounding box area ratios.

Class	Obj (a)	Img (b)	Obj / Img	Mean (c)	Var (c)
aeroplane	1,002	716	1.40	0.2828	0.0715
bicycle	837	603	1.39	0.2644	0.0814
bird	1,271	811	1.57	0.1809	0.0478
boat	1,059	549	1.93	0.1464	0.0477
bottle	1,561	812	1.92	0.0641	0.0173
bus	685	467	1.47	0.2932	0.0668
car	2,492	1,284	1.94	0.1414	0.0537
cat	1,277	1,128	1.13	0.4686	0.0797
chair	3,056	1,366	2.24	0.1215	0.0324
cow	771	340	2.27	0.1778	0.0555
diningtable	800	691	1.16	0.2890	0.0556
dog	1,598	1,341	1.19	0.3603	0.0753
horse	803	526	1.53	0.2918	0.0635
motorbike	801	575	1.39	0.3271	0.0867
person	17,401	9,583	1.82	0.1965	0.0495
pottedplant	1,202	613	1.96	0.1108	0.0419
sheep	1,084	357	3.04	0.1327	0.0391
sofa	841	742	1.13	0.4090	0.0955
train	704	589	1.20	0.3588	0.0689

Class	Obj (a)	Img (b)	Obj / Img	Mean (c)	Var (c)
tvmonitor	893	645	1.38	0.1325	0.0329
Total	40,138	17,125*			

***Table 1**: Per-class dataset statistics. Obj (a): total object instances. Img (b): images containing the class. Mean/Var (c): bounding box area ratio (object area / image area).*

**Total images in the dataset (multi-label, so column b sums to more).*

The dataset exhibits severe class imbalance as shown in figure 1: *person* accounts for 43.3% of all object instances (17,401) and appears in 56% of images, while classes such as *bus* (685) and *train* (704) have roughly 25x fewer instances. The object/image ratio reveals multi-instance patterns: *sheep* averages 3.04 objects per image and *chair* 2.24, indicating these classes frequently appear in groups.

Area ratios reveal significant scale variation across classes. Large-object classes such as *cat* (mean 0.47), *sofa* (0.41), and *dog* (0.36) tend to fill the frame, while *bottle* (0.06) and *potted plant* (0.11) are consistently small. High-variance classes like *sofa* (0.096) and *motorbike* (0.087) appear at diverse scales. These findings motivate the use of class-weighted losses and targeted augmentation in Tasks 3 and 5.

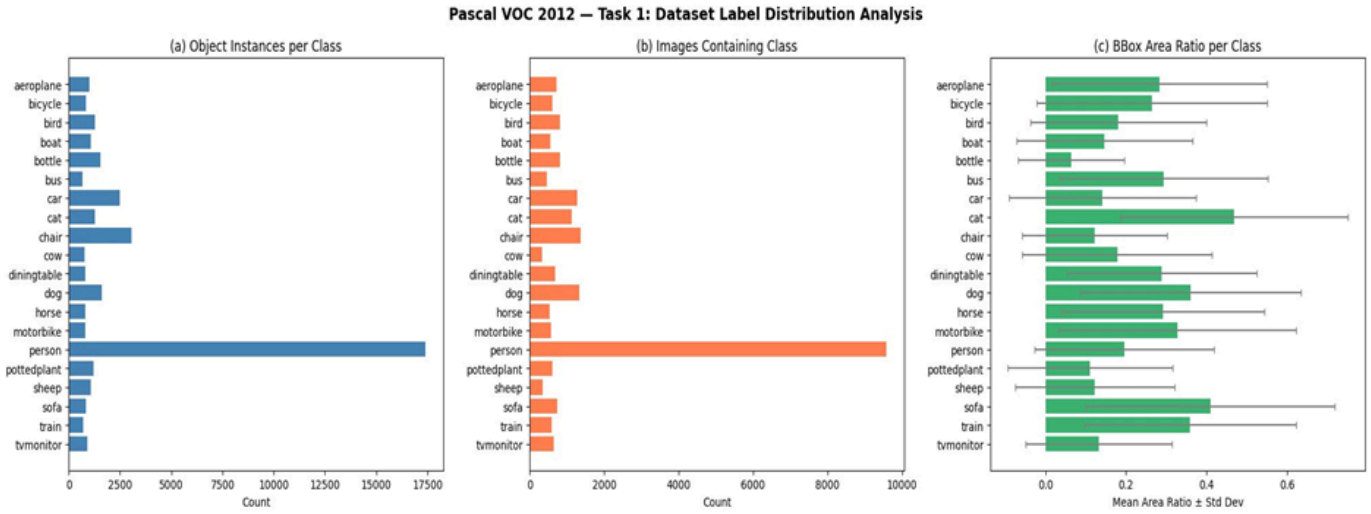

***Figure 1 :** Dataset label distribution: (a) object instance counts, (b) image counts, and (c) mean bounding box area ratio (± std dev) per class.*

Task 2 - Implementation of baseline CNN

Figure 2 illustrates the training and testing (validation) performance over 12 epochs for a baseline multi-label image classification model. As per the task requirements, the model is a ResNet-18 architecture (pretrained on ImageNet) fine-tuned on the Pascal VOC 2012 dataset. The network was

trained using the Adam optimizer and BCEWithLogitsLoss to handle the multi-hot encoded target vectors, utilizing minimal data augmentation (only resizing and normalization).

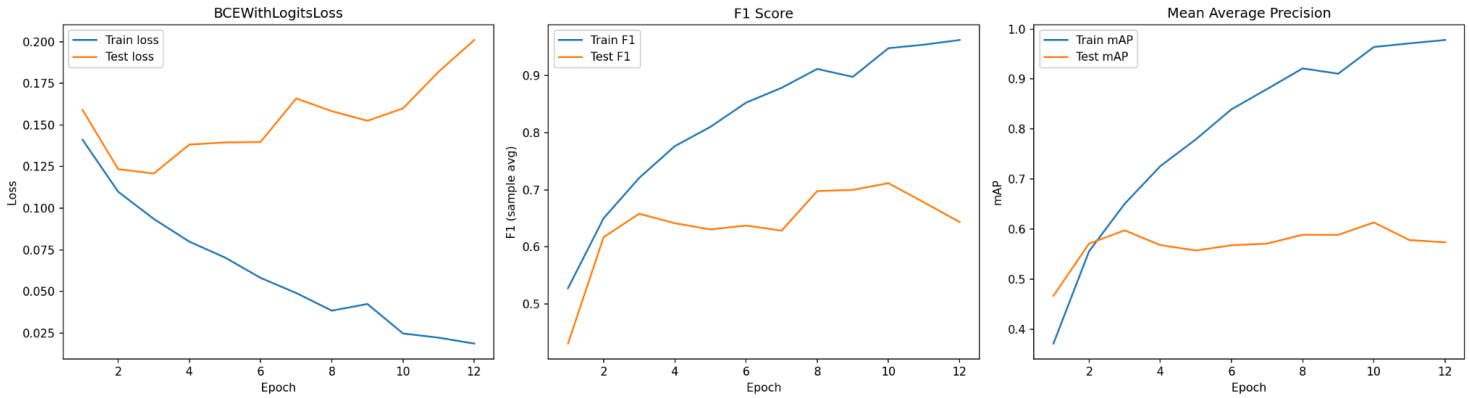

Figure 2: *Training and testing learning curves (BCEWithLogitsLoss, F1 Score, and Mean Average Precision) over 12 epochs for the baseline pretrained ResNet-18 model on the Pascal VOC 2012 dataset.*

The provided plots show how our baseline ResNet-18 model performed over 12 training epochs on the Pascal VOC 2012 dataset. Following the task requirements, we used a pretrained model, the Adam optimizer, and BCEWithLogitsLoss to handle the multi-hot labels, while keeping data augmentation strictly to resizing and normalization.

When looking at the learning curves, a clear pattern emerges across all three metrics: Loss, F1 Score, and Mean Average Precision (mAP). The training metrics show continuous, smooth improvement throughout the entire run. The training loss steadily drops toward zero, while both the training F1 score and mAP climb impressively, reaching near-perfect scores by the final epoch. However, the test metrics tell a different story. They initially improve right alongside the training data but hit a hard ceiling around the third epoch.

This divergence is a classic sign of severe overfitting. After epoch three, the test loss actually starts to climb back up, while the test F1 and mAP scores essentially flatline. This happens because of our specific baseline setup. We are using a highly capable, pretrained ImageNet model, which already knows how to extract strong visual features. Because we are only using minimal data augmentation, showing the network the exact same pixel representations in every single epoch, the model quickly resorts to memorizing the training dataset instead of learning general patterns that apply to new, unseen images.

Ultimately, the baseline model successfully learns the task, but it memorizes the training data much too fast. Based on these curves, the ideal time to stop training would have been right around epoch three, which is where the test loss was at its lowest and the validation metrics had stabilized before degrading.

The baseline model successfully learns the task but overfits early in the training process. The optimum stopping point for this specific configuration would have been at epoch 3, where the test loss was at its lowest and the validation metrics had stabilized.

Task 3 - Architecture comparison

To understand how different network architectures handle this classification task under our baseline constraints, we compared four models: ResNet-18, ResNet-50, MobileNetV3-Large, and EfficientNet-B0. By keeping the training schedule, optimizer, and loss function identical, we can isolate the impact of the architecture itself on performance, efficiency, and generalization.

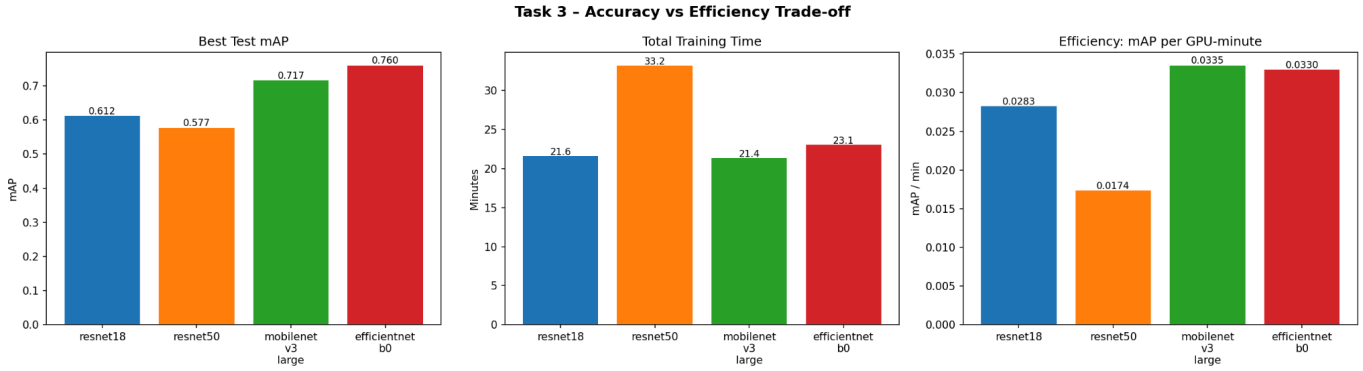

Figure 3: *Comparison of Accuracy vs Efficiency Trade-off in different Architectures*

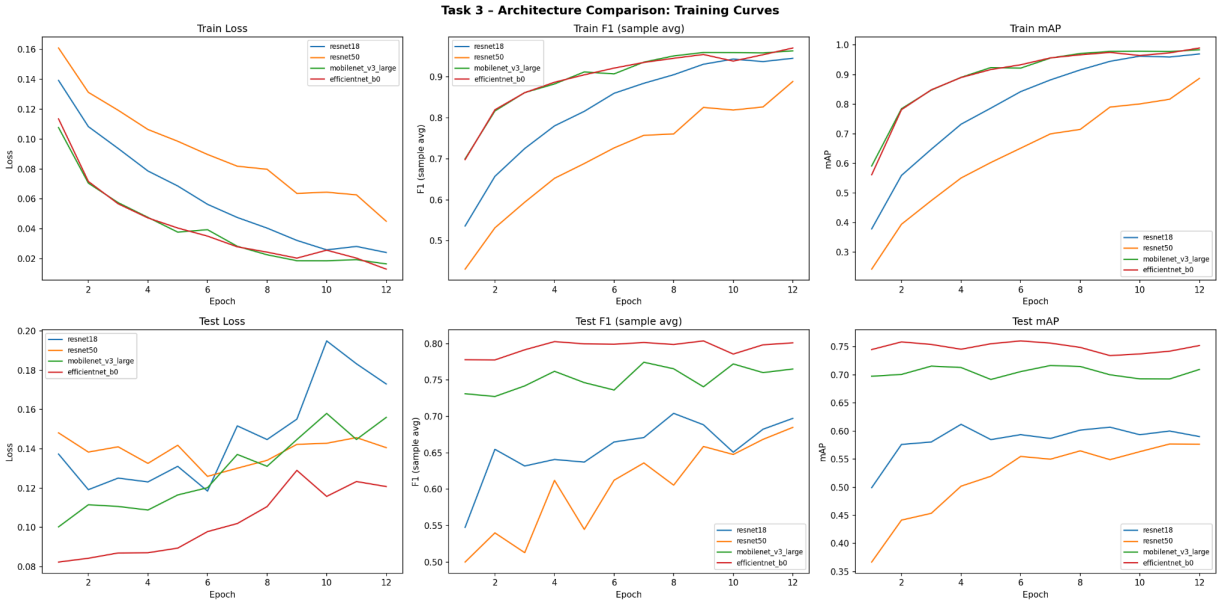

Figure 4: *Train and Test comparison per Model on Loss, F1-value and mAP*

Based on our results, scaling up the depth of the model does not inherently guarantee better performance, especially when training data is effectively limited by a lack of augmentation. When we compare ResNet-18 to its much deeper counterpart, ResNet-50, the deeper model actually performs

worse (Figure 3 and 4). ResNet-18 achieves a peak test mAP of 0.612, while ResNet-50 only reaches 0.577. The deeper architecture of ResNet-50 means it has significantly more parameters, making it more prone to memorizing the training set quickly rather than learning generalized features when data variance is low.

When evaluating the trade-off between accuracy and computational cost in Figure 3, MobileNetV3-Large emerges as the most efficient architecture in our lineup. Looking at the efficiency metric (mAP per GPU-minute), MobileNetV3-Large scores the highest at 0.0335, very closely followed by EfficientNet-B0 at 0.0330. Despite taking slightly less time to train than EfficientNet-B0 (21.4 minutes vs. 23.1 minutes), MobileNet still delivers a highly competitive mAP. Conversely, ResNet-50 is the least efficient model by a wide margin (0.0174 mAP/min), as it requires the longest training time (33.2 minutes) while yielding the lowest overall accuracy.

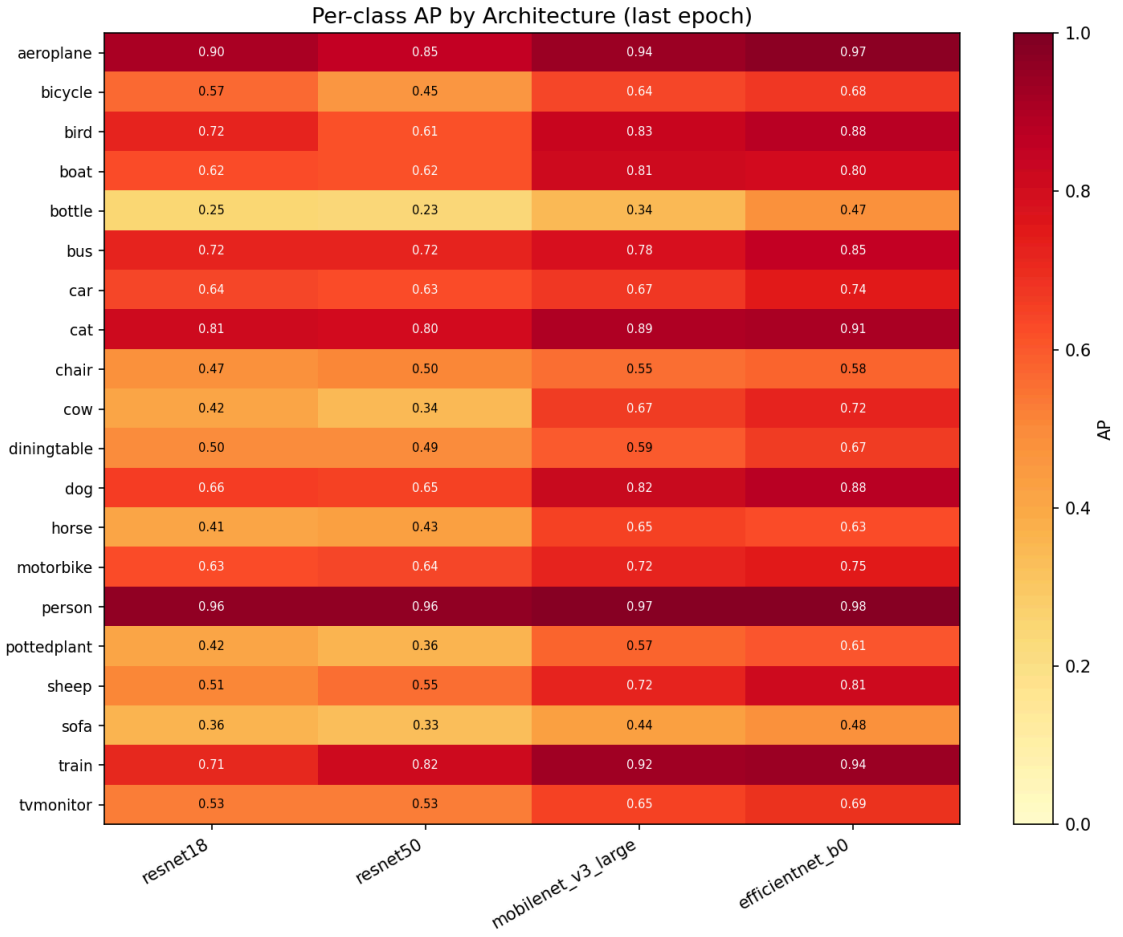

Figure 5: *AP per class and Architecture*

By examining the per-class Average Precision (AP) at the final epoch in Figure 5, we can evaluate how the architectures handle objects of varying scales. In the VOC dataset, classes like "bottle" (mean scale 0.06) and "pottedplant" (mean scale 0.11) are consistently small, making them significantly harder to extract features from compared to large, frame-filling objects like cats (mean scale 0.47) or sofas (mean scale 0.41). Among our tested models, ResNet-50 struggles the most with these challenging small objects. It achieves an AP of just 0.23 for "bottle" and 0.36 for "pottedplant,"

marking the lowest scores across all four architectures for these categories. Conversely, the EfficientNet-B0 architecture demonstrates a much stronger ability to identify small-scale features, effectively doubling the AP for the "bottle" class to 0.47 and improving "pottedplant" to 0.61.

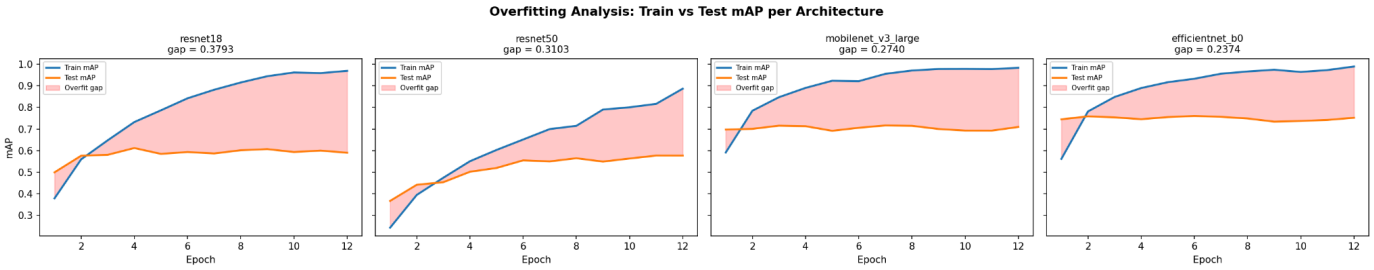

Figure 6: *Overfitting Analysis of mAP in the different Architectures*

Because all models were trained with minimal data augmentation, severe overfitting is a universal issue seen across all four architectures in Figure 6. In every case, the training mAP approaches a perfect 1.0, while the test mAP plateaus early on. However, the severity of the overfitting varies. The older ResNet architectures show the largest "overfit gaps" between their training and testing performance. ResNet-18 suffers the most, with a massive gap of 0.3793. The newer, highly optimized architectures are slightly more robust; EfficientNet-B0 exhibits the smallest overfit gap (0.2374) while simultaneously achieving the highest overall test mAP (0.760). This suggests that modern architectures with advanced regularization features (like squeeze-and-excitation blocks) handle data-constrained scenarios slightly better, though they are not immune to the core problem.

Our comparison of four architectures under limited data augmentation shows that greater depth does not guarantee better results, with ResNet-50 actually performing worse than ResNet-18. While MobileNetV3-Large proved to be the most computationally efficient, EfficientNet-B0 emerged as the strongest overall performer. It achieved the highest test accuracy, best handled small objects, and was the most resilient against the severe overfitting that impacted all the models.

Task 4 - Transfer learning strategies

To optimize our chosen EfficientNet-B0 architecture, we explored five different transfer learning approaches: training entirely from scratch, freezing the pretrained backbone, gradual unfreezing, using discriminative learning rates, and applying a learning rate warmup. By comparing these strategies, we can clearly see the massive impact that initialization and fine-tuning techniques have on convergence speed, overfitting, and overall accuracy.

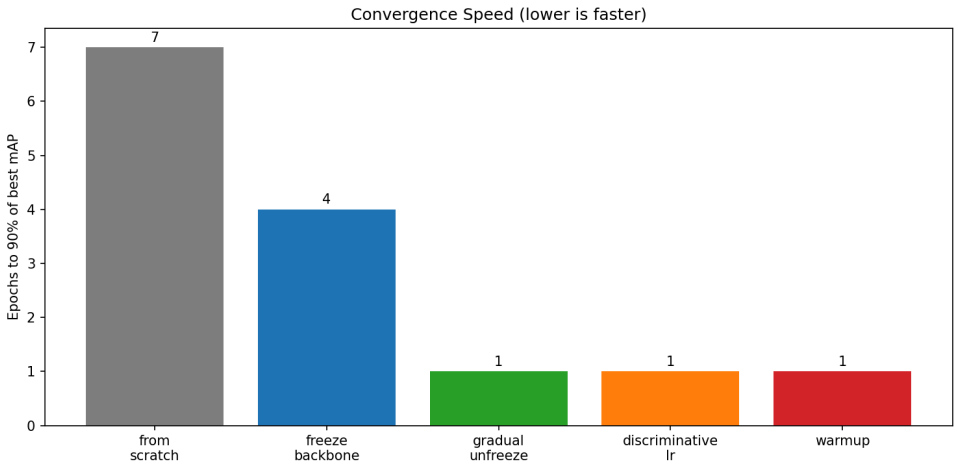

Figure 7: *Comparison of convergence speed between strategies*

As Figure 7 shows, the most immediate takeaway is that training from scratch is highly ineffective for this specific dataset and task. The model initialized from scratch struggles to learn meaningful features quickly, taking 7 full epochs just to reach 90% of its own peak performance. Furthermore, it maxes out at a very poor test mAP of roughly 0.40 by the final epoch. This cleanly demonstrates that utilizing pretrained ImageNet weights is absolutely essential. When we leverage these pretrained weights, the speed at which the models learn accelerates dramatically. The advanced fine-tuning strategies: gradual unfreezing, discriminative learning rates, and warmup are incredibly fast, hitting 90% of their best test mAP within the very first epoch. In contrast, simply freezing the backbone and only training the final classification head is comparatively sluggish, requiring 4 epochs to reach its 90% threshold.

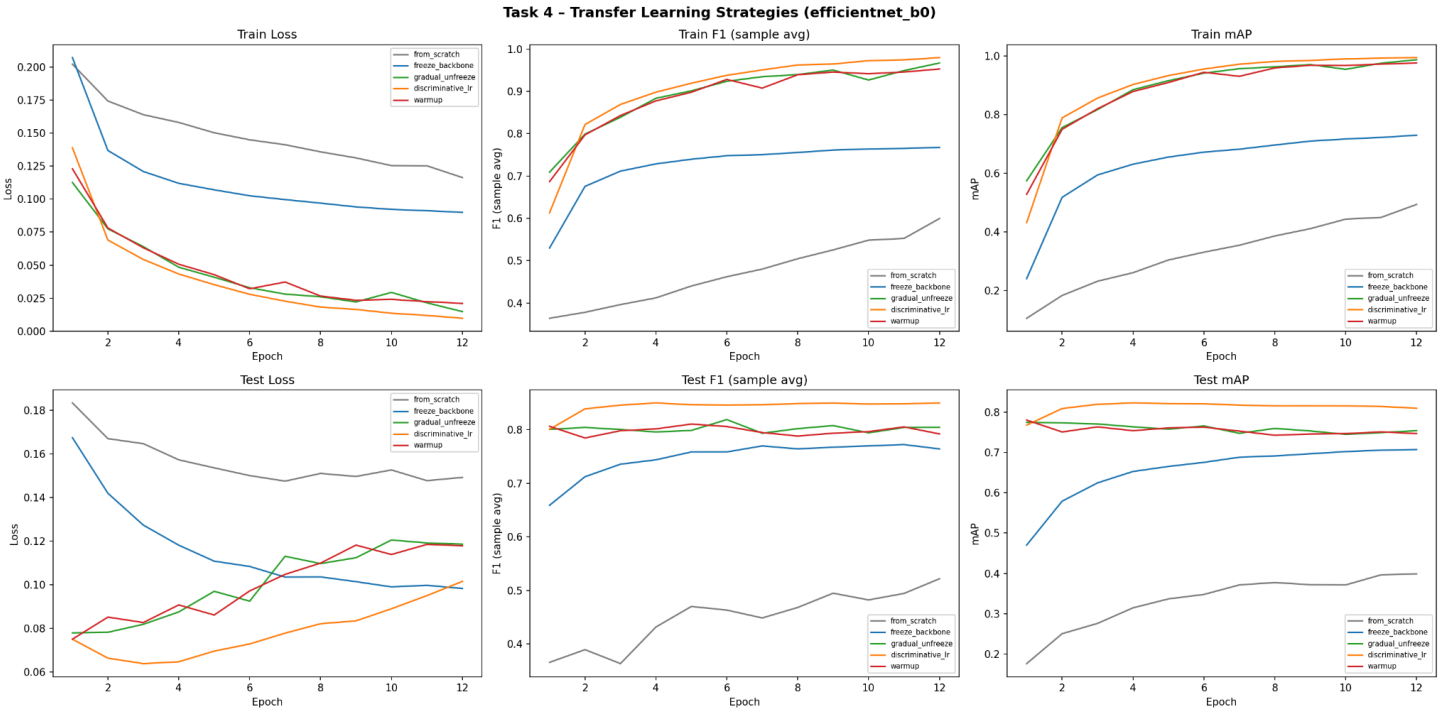

Figure 8: *Overfitting of mAP per strategy*

There is a distinct trade-off between maximizing raw accuracy and controlling the model's tendency to overfit. As shown in figure 8, if perfect generalization to unseen data is the top priority, the standard frozen backbone strategy is the clear winner. Because the vast majority of the network's parameters are locked, it simply cannot memorize the training set, resulting in an incredibly low overfit gap of just 0.0225. The catch is that its overall predictive power is much more limited, capping out at a test mAP of roughly 0.70. On the other hand, the more aggressive fine-tuning methods achieve much higher test accuracy but suffer from larger overfit gaps. For example, gradual unfreezing and warmup show significant overfit gaps of 0.2328 and 0.2290, respectively.

To select the best strategy for our next experiments, we must find the optimal compromise between top-tier accuracy and manageable overfitting. The discriminative learning rate strategy stands out as the best choice. By applying a lower learning rate to the delicate pretrained backbone and a higher one to the newly initialized classification head, it achieves the highest overall test mAP, peaking above 0.80. At the same time, it maintains a slightly tighter overfit gap of 0.1845, which is better than both the gradual unfreezing and warmup approaches. Because it delivers the highest predictive performance while keeping the overfitting relatively in check, discriminative learning rates will be the chosen strategy moving forward.

Task 5 - Data and loss design

By comparing different loss functions, classification heads, and data augmentations, we found that Focal Loss is significantly better at managing unbalanced data than Weighted BCE. In our basic setups, the model using Focal Loss with a linear head reached a higher mean Average Precision of 0.711, compared to 0.694 for the Weighted BCE version. Because Focal Loss automatically adjusts during training to focus more on rare and difficult examples using a modulating factor $(1 - p_t)^{\gamma}$ with $\gamma = 2.0$, it also saves us the effort of manually calculating penalty weights for all twenty categories. All eight configurations used EfficientNet-B0 with discriminative learning rates, standard data augmentation (random resized crop, horizontal flip, and color jitter), and were trained for 12 epochs.

Configuration	Loss	Head	Mixing	Best mAP	Best F1
wbce_linear	Weighted BCE	Linear	—	0.694	0.711
wbce_mlp	Weighted BCE	MLP	—	0.684	0.688
wbce_mlp_mixup	Weighted BCE	MLP	MixUp	0.674	0.619
wbce_mlp_cutmix	Weighted BCE	MLP	CutMix	0.696	0.597
focal_linear	Focal	Linear	—	0.711	0.691
focal_mlp	**Focal**	**MLP**	**—**	**0.726**	**0.680**
focal_mlp_mixup	Focal	MLP	MixUp	0.695	0.589

focal_mlp_cutmix	Focal	MLP	CutMix	0.706	0.600

***Table 2**: Data and Loss Design Results using EfficientNet B0*

When we added a more complex two-layer MLP classification head with BatchNorm, ReLU, and 0.4 Dropout, the results depended entirely on the chosen loss function. Using an MLP alongside Weighted BCE actually lowered our mAP from 0.694 to 0.684. However, when paired with Focal Loss, the MLP performed exceptionally well and achieved the highest overall mAP in our tests at 0.726. This shows that the extra layers in the MLP are very helpful, but only if Focal Loss is there to properly guide the learning process. Without that guidance, the larger model simply memorizes the most common classes faster instead of learning useful patterns.

We originally expected that advanced data mixing techniques like MixUp or CutMix would help the model learn more robust features. Instead, the test data shows they actually hurt performance, consistently lowering both our mAP and F1 scores. For instance, adding CutMix to our best Focal Loss + MLP setup dropped the mAP from 0.726 to 0.706 and the F1 score from 0.680 to 0.600. These augmentations likely failed for two main reasons. First, Pascal VOC is a multi-label problem where each image can contain multiple objects simultaneously; MixUp linearly interpolates both pixel values and label vectors, producing ambiguous soft labels that no longer correspond to any realistic visual configuration. Second, because many objects in this dataset are quite small, CutMix's randomly placed patches likely cover up those tiny objects while the target label still states the object is present, confusing the model.

The per-class frequency versus average precision analysis (Figure 9) reveals that the five worst-performing classes are bottle, potted plant, sofa, chair, and horse, which closely resemble the small-object and high-variance classes identified in Task 1. The five strongest classes are person, cat, aeroplane, train, and dog, confirming that both class frequency and object scale remain the dominant factors influencing classification difficulty. Critically, the Focal Loss + MLP combination narrows the gap between the strongest and weakest classes relative to the baseline BCE, supporting the claim that Focal Loss provides a more balanced gradient distribution across the class spectrum.

This per-class analysis also makes clear that the network does not converge at the same speed for all classes. Rare classes receive far fewer positive gradient signals per epoch, so their per-class classifiers lag behind frequent ones throughout training. The techniques we implemented, Weighted BCE and Focal Loss, directly address this by amplifying the gradient contribution of underrepresented or hard to classify examples. Class-balanced sampling via a WeightedRandomSampler would ensure each mini-batch contains a more uniform distribution of positive examples. Dynamic loss re-weighting could increase the per-class loss weight each epoch for classes whose AP has not improved while decreasing it for the ones that already converged. A two-stage training approach could first train on a class-balanced subset until all classes surpass an AP threshold, then fine-tune on the full imbalanced dataset. Targeted augmentation could apply MixUp or CutMix exclusively to rare class images, avoiding the soft label confusion we observed when applying these techniques globally. Finally, per-class learning rate scaling could assign higher learning rates to the output neurons of slow-converging rare classes.

Ultimately, the combination of Focal Loss and the MLP head emerged as the clear winner. It reached the highest precision of 0.726 while maintaining a solid F1 score of 0.680, and it finished training in

just over 40 minutes. This proves that switching to Focal Loss and adding the MLP provides a meaningful boost in accuracy without significantly increasing the training time compared to our basic models.

We also developed the complete codebase to address the optional bonus improvements, though compute and time constraints prevented us from fully executing these final training loops. Had we been able to run the hyperparameter tuning for our BEST_LR and BEST_BS variables, we expected to see smoother model convergence and a slight bump in our final mean Average Precision (mAP). Furthermore, our implementation of mixed precision training utilizing PyTorch's autocast context and GradScaler was designed to significantly reduce memory consumption. We hypothesized this would have allowed us to increase our effective batch size without running out of VRAM, cutting down our overall training time considerably. Finally, we wrote the logic to tune per-class prediction thresholds, sweeping a range of probabilities to maximize the f1_score for each category instead of relying on a static 0.5 cutoff. We anticipated this specific improvement would have the highest impact on our qualitative results, balancing the macro-F1 metric by significantly reducing false positives for dominant classes and improving recall for underrepresented categories.

Conclusions

In this study, we systematically evaluated CNN architecture design choices for multi-label classification on Pascal VOC 2012 under constrained augmentation settings. Our findings demonstrate that increasing model depth does not inherently improve performance. In fact, ResNet-50 underperformed ResNet-18 despite its higher capacity, highlighting the risk of overfitting in low-variance training regimes.

From the evaluated architectures, EfficientNet-B0 emerged as the strongest overall performer. It achieved the highest test mAP (0.760 in Task 3), showed superior robustness to overfitting with the smallest overfit gap, and handled small-object classes more effectively than traditional ResNet variants. While MobileNetV3-Large offered the best efficiency in terms of mAP per GPU-minute, EfficientNet-B0 provided the best overall balance between accuracy, robustness, and computational cost. Transfer learning proved essential. Training from scratch was highly ineffective, whereas discriminative learning rates delivered the best compromise between convergence speed, peak performance, and manageable overfitting. This confirmed that careful fine-tuning strategy selection is as critical as architecture choice.

Finally, supervision design played a decisive role. Focal Loss significantly outperformed Weighted BCE by improving minority-class performance and balancing gradient contributions across classes. When combined with a two-layer MLP classification head, it achieved the highest overall test mAP (0.726 in Task 5) without increasing computational cost substantially. Global MixUp and CutMix degraded performance in this multi-label setting, probably due to soft-label ambiguity and the presence of small objects. Overall, the combination of EfficientNet-B0, discriminative learning rates, and Focal Loss with an MLP head provided the most effective and balanced solution for this task. These results emphasize that successful multi-label classification depends not only on architecture capacity but on harmonizing model design, fine-tuning strategy, and loss formulation with the statistical properties of the dataset.